

Algorithmique et structure de données



Inès Zribi

Institut Supérieur d'Informatique de Mahdia

Contact : ineszribi@isima.u-monastir.tn

Auditoire : 1^{ère} année LCE



Année Universitaire 2025-2026

Plan du cours

3

1. Algorithme : Notions de base
2. Instructions conditionnelles
3. Instructions itératives
4. Les tableaux
5. Recherche et Tri des tableaux
6. Les types complexes
7. Programmation modulaire

Chapitre 1 : Algorithmme : notions de base

1. Introduction

5

- **L'algorithmique** : la science des algorithmes qui vise à étudier les opérations nécessaires à la réalisation d'un calcul.
- **Un algorithme** :
 - une méthode de résolution d'un problème énoncé sous la forme d'une série d'opérations à effectuer.
 - suite des actions à effectuer pour :
 - ✦ réaliser un traitement donné et/ou
 - ✦ résoudre un problème donné
- Exemples d'algorithmes dans la vie courante :
 - Pour cuisiner : recette
 - Pour assembler un meuble : guide de fabrication

2. Pourquoi un algorithme ?

6

- Pour que la « machine » effectue un travail à notre place:

Informatisation d'un problème.

- Pour mener à bien un traitement sur un ordinateur, il faut :
 1. Concevoir un **algorithme** qui décrit comment le traitement doit être fait
 2. Exprimer l'algorithme sous la forme d'un **programme** dans un langage de programmation adéquat (traduire cet algorithme à l'aide d'**un langage de programmation** connu par l'ordinateur)
 3. Faire en sorte que l'ordinateur exécute le programme : **la compilation**

2. Pourquoi un algorithme ?

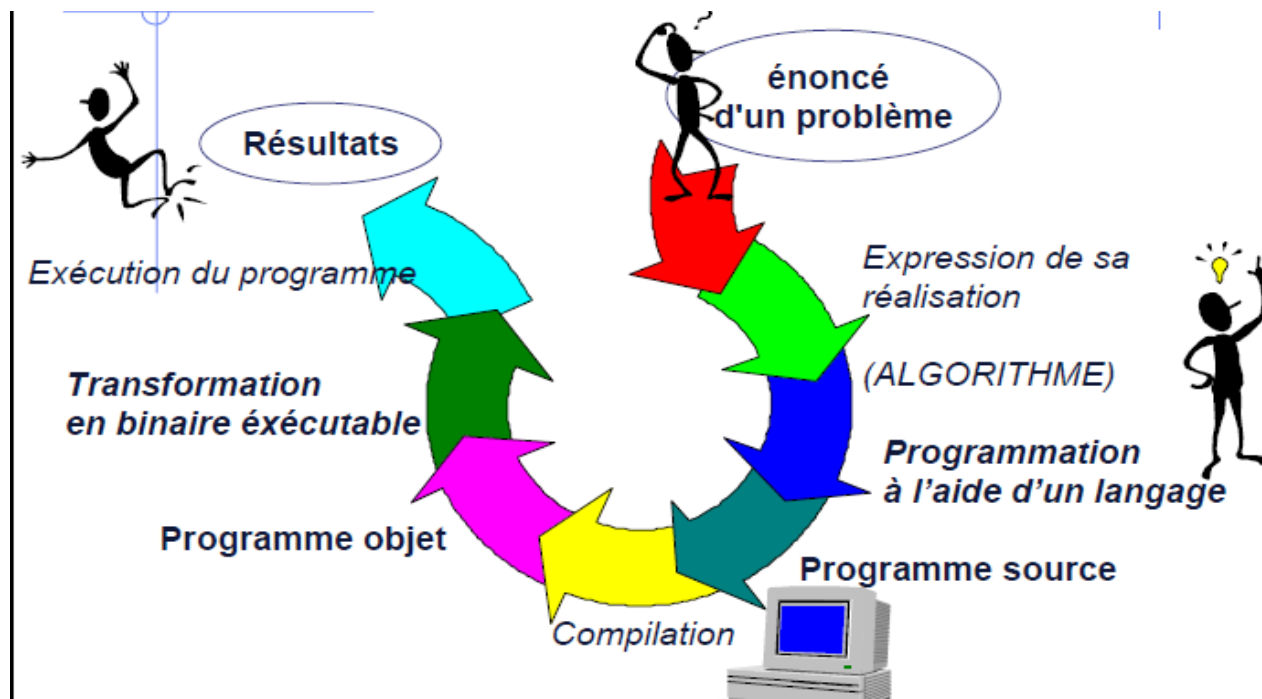
7

- **Programme :**

- codage d'un algorithme afin que l'ordinateur puisse exécuter les actions décrites
 - doit être écrit dans un langage **compréhensible** par l'ordinateur (Exemple : Assembleur, Python, Basic, C, Fortran, Pascal, Cobol, JAVA, ...)
-
- Un programme est donc **une suite ordonnée d'instructions** élémentaires codifiées dans un langage de programmation.

Étapes de réalisation d'un programme

8



3. Éléments de base d'un algorithme

9

En général, la réalisation d'un algorithme se base sur 2 étapes :

- Analyse et compréhension du problème : étudier les données fournies et les résultats attendus.
- Résolution du problème : trouver les structures de données adaptées (déclaration des **variables et constantes**) ainsi que l'enchaînement des actions (**instructions**) à réaliser pour passer des données aux résultats.
 1. **Variables et constantes** en précisant leurs **types**
 2. **Instructions** :
 - ◆ Instruction d'affectation
 - ◆ Instruction d'entrée
 - ◆ Instruction de sortie

3. Éléments de base d'un algorithme : variable et constante

10

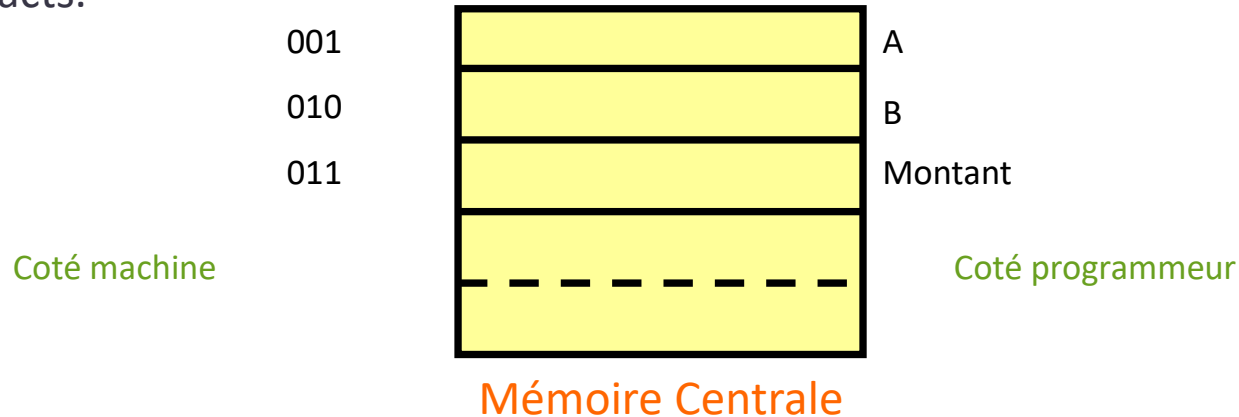
Un algorithme utilise des **objets** qui peuvent être des variables ou des constantes.

- Chaque objet à un rôle qui consiste à mémoriser une valeur particulière significative :
 - **La variable** qui contient une valeur appelée à être modifiée au cours de l'algorithme.
 - **La constante** qui retient une valeur qui ne change pas au cours de l'algorithme
- Un objet est parfaitement défini s'il est parfaitement identifié, typé et renseigné

Variable

11

- Une **variable** est un dispositif de mémorisation d'une valeur à laquelle on accède à tout moment et on change.
- Les variables :
 - Servent à « nommer » des emplacements ou adresses de la mémoire.
 - Permettent de manipuler des valeurs sans connaître leurs emplacements exacts.



Variable

12

- Déclaration d'une variable :

- En algorithmme :

Variables identificateur: type

- identificateur: signification attribuée à la variable

- type : réel, entier, caractère, etc.

- ✦ Exemples :

Variables

x, y : entier

prix, salaire : réel

s : caractère

Variable : identificateur

13

- Un **identificateur** (ou nom d'une variable ou constante) est une suite de caractères parmi :
 - les lettres (minuscules ou majuscules, mais non accentuées),
 - les chiffres,
 - le "blanc souligné" (_)
- Le premier caractère d'un identificateur ne peut pas être un chiffre.
- Exemples :
 - `var1 tab_23 _deb main2 char1` : identificateurs valides
 - `1i i:j char main tab-23-deb` : identificateurs non valides.
- Il est déconseillé d'utiliser « _ » comme premier caractère d'un identificateur.
- **ATTENTION : Les majuscules et minuscules sont différenciées.**

Variable : type d'une variable

14

- Le **type d'une variable** permet :
 - De savoir quel est l'espace mémoire occupé par une variable
 - De connaître les opérations autorisées sur la variable
- **Type numérique :**
 - Entier
 - ✧ 1 6 9685 -25
 - Réel
 - ✧ 1.55 -6.98 0.00005
- **Type caractère :** lettres, chiffres, symboles, etc. placés entre deux ''
 - 'c' 'A' 'b' '+' '1'
- **Type chaîne de caractères :** une suite de caractères
 - "bonjour " "enfant"
- **Type logique ou booléen :** deux valeurs VRAI ou FAUX

Constante

15

- Déclaration d'une constante :

- Syntaxe :

Constante identificateur_const = valeur

- ✦ identificateur_const : signification attribuée à la constante
 - ✦ valeur : contenu de la constante durant l'algorithme

- Exemple :

- ✦ Constante pi = 3,14

- **Remarques :**

- ✦ On ne peut pas attribuer une expression à une constante :
identificateur_const = expression
 - ✦ Exemple : PI = 3 + 5

3. Éléments de base d'un algorithme : les instructions

16

1. Instruction d'affectation
2. Instruction de sortie
3. Instruction d'entrée

Instruction d'affectation

17

- Rôle: mettre une valeur dans un emplacement mémoire désigné par son identificateur (affecter une valeur à une variable).
- Syntaxe : **variable ← expression**
- Exemples :

Instruction	Exemples
identificateur ← valeur	Note ← 15
identificateur1 ← identificateur2	Note1 ← Note2
identificateur ← expression	Moyenne ← (Note1*2 + Note1)/3
identificateur ← constante	Moyenne ← PI

Expressions et opérateurs

18

- Une **expression** est une combinaison **d'opérateurs** et **d'opérandes** (variables, constantes ou encore littéraux). Une expression génère toujours un résultat d'un **type bien défini** qu'on appellera **le type de l'expression**.
 - Expression simple:
 - une seule variable ou une constante.
 - PI //constante
 - Taux //variable
 - Expression complexe :
 - Plusieurs expressions simples avec des opérateurs
 - $2+8$
 - $8+(5*\text{taux})+(\text{taux}*\text{taux})/\text{cout}$
 - $2*PI$

Expressions et opérateurs

19

- Opérateurs arithmétiques :

Opérateurs multiplicatifs

Algorithme	Signification
exp1 * exp2	retourne le résultat de la multiplication
exp1 DIV exp2	retourne le résultat de la division
exp1 MOD exp2	retourne le résultat du modulo (reste de la division entière)
exp1**n	retourne la valeur de exp1 puissance n

Opérateurs additifs

Algorithme	Signification
exp1 + exp2	retourne le résultat de l' addition
exp1 - exp2	retourne le résultat de la soustraction

Expressions et opérateurs

20

- Opération arithmétique et affectation combinées :

Algorithme	Signification
var ← exp	Affectation directe
var ← var * exp	Affectation suite à la multiplication
var ← var DIV exp	Affectation suite à la division
var ← var MOD exp	Affectation suite au modulo
var ← var + exp	Affectation suite à l'addition
var ← var - exp	Affectation suite à la soustraction

Expressions et opérateurs

21

- Opérateurs de comparaison:

Opérateur	Traduction	Syntaxe	Résultat
<	Inférieur	exp1 < exp2	retourne VRAI si exp1 est inférieur à exp2 sinon FAUX
>	Supérieur	exp1 > exp2	retourne VRAI si exp1 est supérieur à exp2 sinon FAUX
<=	Inférieur ou égal	exp1 <= exp2	retourne VRAI si exp1 est inférieur ou égal à exp2 sinon FAUX
>=	Supérieur ou égal	exp1 >= exp2	retourne VRAI si exp1 est supérieur ou égal à exp2 sinon FAUX
=	Égalité	exp1 = exp2	retourne VRAI si exp1 est égal à exp2 sinon FAUX
<>	Inégalité	exp1 <> exp2	retourne VRAI si exp1 est différent à exp2 sinon FAUX

Expressions et opérateurs

22

- **Opérateurs logiques :**

- Les opérateurs logiques, permettent de combiner le résultat de plusieurs expressions de comparaison en une seule expression logique.
 - ✦ $(a < b)$ **ET** $(c > d)$
 - ✦ $(a = b)$ **OU** $(c >= d)$
- Comme pour les expressions de comparaisons, les expressions logiques renvoient une valeur booléenne (FAUX, VRAI).
- Les opérateurs logiques sont évalués de gauche à droite :
 - exp1 **ET** exp2 retourne le résultat du ET logique
 - exp1 **OU** exp2 retourne le résultat de OU logique
 - **NON** exp retourne le résultat du complément logique

Expressions et opérateurs

23

- Opérateurs logiques :**

Valeur		ET logique	OU logique	Complément logique de exp1
exp1	exp2	exp1 ET exp2	exp1 OU exp2	NON exp1
Faux	Faux	Faux	Faux	Vrai
Faux	Vrai	Faux	Vrai	Vrai
Vrai	Faux	Faux	Vrai	Faux
Vrai	Vrai	Vrai	Vrai	Faux

Expressions et opérateurs

24

- **Exercice :**

Soit les deux affectations suivantes :

- $X \leftarrow 3$
- $B \leftarrow 5$

Quel est le résultat des expressions suivantes ?

- $X = 2 \text{ OU } B = 1$
- $((X = 3) \text{ OU } (X \geq 2)) \text{ ET } B > 0$
- $((X < 3) \text{ OU } (X \geq 2)) \text{ ET } B > 0$
- $((X = 3) \text{ ET } (X \geq 2)) \text{ OU } B > 0$

Expressions et opérateurs

25

- **Priorité et associativité des opérateurs :**
 - Lors de l'évaluation des différentes parties d'une expression, les opérateurs respectent certaines lois de priorité et d'associativité.
 - Exemples :
 - La multiplication a la priorité sur l'addition
 - La multiplication et l'addition ont la priorité sur l'affectation.

Expressions et opérateurs

26

- **Priorité et associativité des opérateurs :**

- La priorité est décroissante de haut en bas dans le tableau. Dans une expression, les parenthèses forcent la priorité.

Priorité	Opérateurs
1	()
2	+ - NON
3	* DIV MOD
4	+ -
5	< <= > >=
6	= <>
7	ET
8	OU

Instruction de sortie

28

- **Rôle:** permet de restituer une valeur. Généralement, ça consiste à afficher sur l'écran
- **Syntaxe:**
 1. écrire (**valeur**)
Ex: écrire(4)
 2. écrire (**variable**)
Ex: écrire(Note)
 3. écrire (**expression**)
Ex: écrire ("La moyenne=", (Note1+Note2) DIV 2)
- **Remarque:** **écrire(Note)** n'est pas la même chose que **écrire("Note")**

Instruction d'entrée

29

- **Rôle** : Permet d'introduire une donnée au programme. Généralement, on tape la valeur. À l'exécution de cet ordre, l'ordinateur demande à l'utilisateur de lui communiquer par l'intermédiaire du clavier ou de la souris, une valeur.
- L'information saisie doit être cohérente avec le type de la variable.
- **Syntaxe** : lire(variable)
Ex: Lire(Note)
- **Effet** :
 - ✦ À la rencontre de cette instruction, l'ordinateur arrête l'exécution du programme et attend qu'on tape une valeur.
 - ✦ On termine la saisie en appuyant sur la touche Entrée.
 - ✦ La valeur qu'on tape est affectée à la variable lue

3. Éléments de base d'un algorithme : Les commentaires

30

- Un commentaire est une ligne ou une suite de lignes ignorées par le compilateur.
- Il sert pour expliquer le rôle des instructions qui le suivent.
- Exemple :

// Ceci est un commentaire sur une seule ligne

- On ne peut pas imbriquer des commentaires.
- Quand on met en commentaire un morceau de programme ou algorithme, ce dernier ne sera pas exécuté.

4. Construire un algorithme

31

En général, un algorithme comprend 4 parties :

1. La **déclaration des variables et constantes** nécessaires aux traitements
2. Au début de l' algorithme, **l'initialisation** des variables : affectation de valeurs initiales aux variables déclarées
3. Le **traitement** des **différentes séquences ordonnées** de l'algorithme :
 - ◆ Réalisation d'actions élémentaires ou structurées
 - ◆ Appel de procédures ou de fonctions externes : « sous-traitement » de portée générale qui sont réutilisés dans plusieurs algorithmes différents
4. À la fin de l'algorithme, la **sortie des résultats** et l'arrêt de l'algorithme

4. Construire un algorithme

32

Algorithme : *nom_algo*

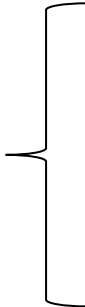
Définition des types

Déclaration des variables et constantes

Début

La suite des instructions

Fin



```
// commentaire 1
Instruction d'affectation
// commentaire 2
Instruction d'entrée
Instruction de sortie
...
```

Exemple

33

- Calcul de l'intérêt et de la valeur acquise par une somme placée pendant un an dans une banque.
- L'énoncé du problème présente :
 - Les données fournies : deux nombres représentant les valeurs de la somme placée et le taux d'intérêt.
 - SI = somme initiale
 - T = taux d'intérêt (ex: 3 pour 3%)
 - Les résultats désirés : deux nombres représentant l'intérêt fourni par la somme placée ainsi que la valeur obtenue après placement d'un an.
 - ✦ Après avoir demandé à votre banquier
 - I = intérêts = $SI \cdot T / 100$
 - SF = somme finale = $SI + I$

Exemple

34

- Formalisation de l'algorithme :

Algorithme : calcul intérêt et somme finale

Variables

SI, I, SF : réel

Constante T = 3

Début

écrire("Donner la somme initiale")

lire(SI)

I ← SI * T DIV 100

SF ← SI + I

écrire (I)

écrire (SF)

Fin

Déclaration des variables

Déclaration des constantes

Instructions

5. Règles à respecter pour l'écriture d'un algorithme

35

1. Un algorithme doit décrire précisément le traitement qu'il doit exécuter et s'assurer que **tous les cas de figures possibles** ont bien été prévus.
2. Le concepteur d'un algorithme doit donc **affiner** ce dernier jusqu'à ce que **les étapes** puissent être écrites à l'aide d'un langage de programmation.
3. Un algorithme est défini **sans ambiguïté**
4. Il se termine après un **nombre fini d'opérations**

Exercices

36

- **Exercice 1 :**

Soient deux valeurs, contenues dans A et B. Écrire un algorithme qui consiste à échanger les valeurs des deux variables.

Exemple : en entrée, nous avons la valeur de A = 2 et celle de B = 4. En sortie, on doit avoir A = 4 et B = 2.

- **Exercice 2 :**

On désire fournissant un nombre de jours, avoir le nombre d'années, de mois et de jours auquel cela correspond.

On utilisera pour ce calcul, des données théoriques de 365 jours et des mois théoriques de 30 jours, la saisie de la donnée et l'édition des résultats se feront sous la forme :

Nombre de jours : 400

Correspondent à 1 année(s), 1 mois et 5 jour(s).

Écrire l'algorithme correspondant.

- **Exercice 2 :**

On désire fournissant un nombre de jours, avoir le nombre d'années, de mois et de jours auquel cela correspond.

On utilisera pour ce calcul, des données théoriques de 365 jours et des mois théoriques de 30 jours, la saisie de la donnée et l'édition des résultats se feront sous la forme :

Nombre de jours : 400

Correspondent à 1 année(s), 1 mois et 5 jour(s).

Écrire l'algorithme correspondant.